Prefill Is the Bottleneck: An Empirical Study of CPU-Only On-Device LLM Latency and One Lever That Exploits It

Bapista Khan
Collab-Foundry / NeuronAI
bapistakhan@gmail.com

July 2026

Abstract

Sovereign, offline-first AI increasingly runs large language models (LLMs) on commodity CPUs with no GPU, yet latency intuition remains grounded in GPU serving, where autoregressive *decode* dominates. We instrument end-to-end inference of a quantized 7B model (Qwen2.5-7B, Q4) on a single CPU-only machine (AMD Ryzen 7 8845HS, 8 cores / 16 threads, 14 GiB) and sweep prompt length from 55 to 2,336 tokens (5 repetitions per point). *Prefill*—reading the prompt—grows from 38% of end-to-end latency at 55 tokens to **95%** at 2,336 tokens, crossing 90% for prompts beyond ∼1,300 tokens. Prefill time scales with prompt length while decode stays roughly flat (∼11–12 tok/s), so end-to-end latency on CPU is governed by *context length*, not output length—inverting the GPU-era assumption. Error bars are tight (prefill fraction 91±0.1% at 1,299 tokens), and the effect is not model-specific: Llama-3.1-8B and Gemma2-9B also sit at 95–96% prefill at ∼1,300 tokens. We further find prefill speed is *non-monotonic* in quantization: the 8-bit build prefills fastest and the intermediate K-quant slowest, inverting the "quantize-down-for-speed" heuristic on CPU. Both effects reproduce on a second CPU vendor (Intel Comet Lake), where prefill dominance is if anything stronger. Finally, we translate the measurement into a deployment lever: in a controlled test, gating a second "review/verify" model pass—the costliest optional stage in a typical agent pipeline—cuts latency 2.4× with *no* measurable quality loss under an order-swapped LLM judge, showing the prefill finding pays off in practice. We release all harnesses and raw data.

1 Introduction

Offline-first and *sovereign* LLM deployment—driven by privacy, data governance, connectivity, and cost—has made CPU-only edge inference a real operating regime. The NeuronAI system this study measures runs a fleet of 7–9B quantized models entirely on CPU on a 14 GiB mini-PC-class machine, with no GPU offload.

Most latency intuition, however, is inherited from GPU serving. There, prefill is highly parallel and cheap relative to the long sequential tail of decode, so decode dominates and optimization targets it. *On CPU-only edge hardware, is that still true?*

We answer empirically. Our contributions, stated as measured facts:

1. An open harness that decomposes on-device LLM latency into prefill and decode using the runtime's own timing counters (Section 3).

2. The finding that on CPU-only hardware, prefill **exceeds 90%** of end-to-end latency at realistic prompt lengths (≥1,300 tokens) and reaches 95% at 2,333 tokens (Section 4).

3. A characterization of the prefill fraction as a monotonically rising function of prompt length, and the finding that prefill *speed* is non-monotonic in quantization—the 8-bit build is fastest—making quant choice a prefill-side lever (Section 4.4).

4. A controlled demonstration that one prefill-motivated lever—gating an extra "review/verify" model pass to only the hardest queries—cuts latency 2.4× with no measurable quality loss, translating the measurement into a deployment win (Section 6).

Scope. The main sweep is deliberately bounded to *single-machine, CPU-only inference of one quantized 7B model*, with a cross-family spot-check at 7B–9B (Section 4.3), a Q4/Q5/Q8 quantization sweep (Section 4.4), and a cross-vendor replication on a second CPU (Intel; Section 4.5). We do not study batched multi-user serving, GPU offload, or non-x86 (ARM) hardware; those are future work (Section 7).

2 Background & Related Work

Prefill vs. decode. LLM inference has two phases. *Prefill* processes the entire input prompt in parallel, building the KV cache; its cost grows with prompt length. *Decode* then emits output tokens one at a time, each step reading the KV cache—sequential and typically memory-bandwidth bound.

Why the balance differs on CPU. On a GPU, thousands of parallel lanes hide prefill's compute, so decode's sequential tail dominates end-to-end latency. A CPU has far less parallelism to hide prefill's per-token matrix work, so we hypothesize that as prompts grow, prefill—not decode—comes to dominate. Our data confirms this directly (Section 4).

Related work. The prefill/decode split underpins modern GPU serving. Systems work manages KV memory [7] and disaggregates the two phases across machines [12, 19] or interleaves chunked prefills with decode to tame the throughput–latency tradeoff [1]. These target GPU clusters and batched multi-tenant serving; our regime is the opposite corner—single-user, single-machine, CPU-only—where the same phase asymmetry manifests very differently. On-device and edge inference is an active area [16], spanning limited-memory flash offload [2], smartphone inference [17], and sub-billion on-device models [10]; we add a direct latency *decomposition* on commodity CPU. Quantization reduces model footprint and cost [4, 5, 9, 15]; we treat 4-bit quantization as the operating context and find quant choice is itself a prefill-side lever (Section 4.4). We build on CPU-capable runtimes [6, 14].

Spending compute adaptively. Our deployment lever (Section 6) relates to work that spends compute by difficulty. Cascades and routers send easy queries to cheaper models [3, 11]; we instead keep a fixed ≥7B model and route *pipeline effort*, motivated by the prefill cost model. Adaptive-computation methods exit the decoder early per token [13]; we skip whole optional stages per request. This is orthogonal to decode-side acceleration such as speculative decoding [8], and we use an LLM judge [18] to check the lever causes no quality regression.

3 Methodology

Platform. Table 1 lists the machine under test. The integrated Radeon 780M GPU is *not* used; all inference is on CPU.

Table 1: Measurement platform. GPU present but unused; all inference CPU-only.

Component	Value
CPU	AMD Ryzen 7 8845HS (Zen 4)
Cores / threads	8 / 16
RAM	14 GiB (shared APU memory)
GPU	Radeon 780M (unused)
OS / kernel	TUXEDO OS, Linux 6.17.0
Runtime	Ollama 0.19.0 (llama.cpp)
Model	Qwen2.5-7B, Q4_K_M (≈4.6 GB), ctx 4096

Table 2: Phase breakdown at a representative 1,299-token prompt. Qwen2.5-7B (Q4), CPU-only, mean of 5 runs.

Model	Params	Quant	Prefill (s)	Decode (s)	Prefill %
Qwen2.5-7B	7B	Q4	28.5	2.8	91

Instrumentation. We drive the runtime's completion endpoint (`/api/generate`, non-streaming) and read its native timing counters: `prompt_eval_duration` (*prefill*) and `eval_duration` (*decode*), with `prompt_eval_count` / `eval_count` giving token counts. This measures prefill *directly* rather than via a time-to-first-token proxy. Output is capped at `num_predict`=64, temperature 0, `num_ctx`=4096. The model is warmed once and kept resident (load time excluded). Each prompt carries a unique prefix nonce so the runtime's prompt-prefix cache cannot shortcut the prefill. We sweep six prompt lengths (51–2,333 tokens) built from neutral filler text.

Reproducibility. Each prompt length is run 5 times; we report mean±sample std. Harness and raw per-run JSON are released (Section 8).

4 Results

4.1 Headline: prefill dominates

At a representative operational prompt length (1,299 tokens, near NeuronAI's ∼1,525-token production prompt), prefill is **91%** of end-to-end latency (Table 2): 28.5 s of prefill versus 2.8 s of decode (mean of 5 runs; std <0.15 s).

4.2 Prompt-length sweep (main result)

Table 3 and Figure 1 show the core finding: the prefill fraction rises *monotonically* with prompt length, 38%→95%, and crosses 90% past ∼1,300 tokens. Prefill time scales roughly linearly with prompt length; decode

Table 3: Prompt-length sweep, mean±std over 5 runs. Qwen2.5-7B (Q4), CPU-only, output capped at 64 tokens. Measured 2026-07-14.

Prompt (tok)	Prefill (s)	Decode (s)	Prefill %
55	0.57±0.05	0.95±0.10	38±4.2
159	2.71±0.03	2.83±0.19	49±1.6
367	7.22±0.09	2.83±0.11	72±0.7
678	14.19±0.09	3.20±0.02	82±0.1
1,299	28.47±0.12	2.75±0.01	91±0.1
2,336	53.66±0.29	2.64±0.30	95±0.5

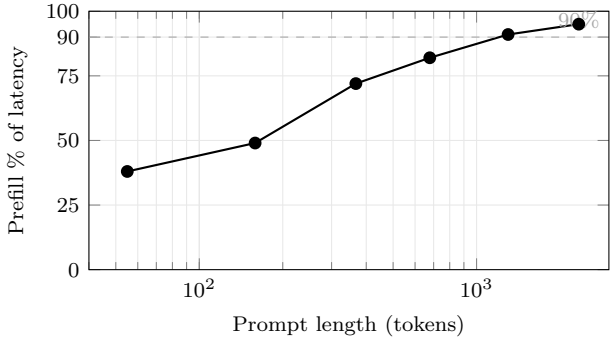

Figure 1: Prefill fraction of end-to-end latency vs. prompt length (CPU-only, Qwen2.5-7B Q4). Rises monotonically 35%→95%, crossing 90% past ∼1,300 tokens.

time is nearly constant (output is fixed at 64 tokens). Prefill throughput drifts down with length (∼50→44 tok/s over the realistic range) as per-token attention cost grows, while decode holds at ∼11–12 tok/s. Variance is low: at long prompts the prefill fraction has std ≤0.5 percentage points.

4.3 Model size: prefill dominance is stable across families

To check that prefill dominance is not a Qwen artifact, we repeat the measurement at a fixed ∼1,300-token prompt across three model families spanning 7B–9B (Table 4; 3 runs each, one model resident at a time). The prefill fraction stays in a tight **92–96%** band and prefill throughput declines gently with size (47.6 → 43.5 tok/s). Because the families differ in architecture and tokenizer (hence the slightly different token counts), this is a size-and-architecture comparison rather than a controlled scaling curve; the invariant it establishes is that prefill dominance is a property of the CPU-only regime, not of any one model.

Table 4: Model-size sweep at a fixed ∼1,300-token prompt, mean±std over 3 runs. Q4, CPU-only, output capped at 64 tokens. Different families ⇒ a size+architecture comparison.

Model	Params	Prefill (s)	tok/s	Prefill %
Qwen2.5-7B	7B	27.37±0.20	47.6	92±0.1
Llama-3.1-8B	8B	29.43±0.16	43.5	96±0.1
Gemma2-9B	9B	29.39±0.57	43.6	95±0.2

Table 5: Quantization sweep: same model (Qwen2.5-7B-Instruct), fixed ∼1,300-token prompt, mean±std over 5 runs, CPU-only. Prefill speed is non-monotonic in footprint; ordering confirmed under reversed run order.

Quant	Footprint	Prefill (s)	tok/s	Prefill %
Q4_K_M	∼4.7 GB	28.03±0.10	46.3	91
Q5_K_M	∼5.4 GB	37.35±0.04	34.8	95
Q8_0	∼8.1 GB	21.59±1.60	60.4	90

4.4 Quantization: prefill speed is non-monotonic in footprint

Holding the model fixed (Qwen2.5-7B-Instruct) and varying only weight precision reveals a counterintuitive result (Table 5; N=5). Prefill throughput is *not* monotonic in model size-in-bytes: **Q8_0**, the *largest* build (∼8.1 GB), prefills *fastest* (∼60 tok/s), while the intermediate **Q5_K_M** is *slowest* (∼35 tok/s) and **Q4_K_M** sits between (∼46 tok/s). On compute-bound CPU prefill the *dequantization scheme*, not the byte footprint, dominates: Q8_0's near-trivial dequant (a single scale per block) beats the more elaborate block-structured unpacking of the K-quants. The ordering is robust—it reproduces when the sweep is run in reverse order (Q8→Q4), ruling out a thermal/warm-up artifact.

Implication. On CPU, quantizing *harder* to shrink a model can *cost* prefill latency: Q8_0 here is simultaneously the latency-fastest and (by bit-width) the highest-fidelity build, trading only memory footprint. This inverts the "quantize down for speed" heuristic inherited from memory-bandwidth-bound decode, and makes quantization choice a genuine prefill-side lever on edge hardware.

4.5 Generalization to a second CPU vendor

To test whether these findings are specific to AMD/Zen 4, we repeat the prompt-length and quantization sweeps on a machine of a different vendor and microarchitecture: an **Intel Core i7-10700T** (Comet

Table 6: Second CPU vendor: Intel Core i7-10700T (Comet Lake), CPU-only, Qwen2.5-7B-Instruct, ∼1,400-token prompt, mean±std over 5 runs. Both prefill dominance and the non-monotonic quant ordering reproduce.

Quant	Prefill (s)	tok/s	Prefill %
Q4_K_M	83.54±0.46	16.8	93
Q5_K_M	105.23±1.67	13.3	98
Q8_0	71.77±0.53	19.5	95

Lake, 8C/16T, 35 W TDP, 2020), CPU-only, same Ollama/llama.cpp runtime and models (N=5). Both findings carry over.

Prefill still dominates—more so. On the slower Intel part the prefill fraction is *higher* at every prompt length—51% at 121 tokens, 85% at 453, 93% at 1,403, and 96% at 2,439—than on the Ryzen, consistent with prefill's compute-bound cost growing faster than decode as the CPU slows. Prefill dominance is thus not a property of one vendor; if anything, weaker CPUs are *more* prefill-bound.

The non-monotonic quant ordering reproduces. At a fixed ∼1,400-token prompt, Q8_0 again prefills *fastest* and Q5_K_M *slowest* (Table 6)—the same ordering as on AMD. "Q8 is fastest on CPU" is therefore a property of the dequantization scheme, not of one microarchitecture. Absolute prefill throughput is ∼2.7× lower than the Ryzen (16.8 vs. 46.3 tok/s at Q4), as expected for a 2020 low-power part.

5 Analysis & Discussion

Why prefill dominates on CPU. Prefill is compute-bound over every prompt token and, on a CPU, has little parallelism to hide that work; its cost scales with prompt length. Decode is a fixed, memory-bandwidth-bound step per output token. With output held constant, growing the prompt inflates only the prefill term, driving the fraction toward 1. The declining prefill throughput (88→43 tok/s) is consistent with attention's growing per-token cost as the sequence lengthens.

Implications for sovereign/edge deployment. On CPU, the levers that matter are the ones that shrink or reuse the prompt, not the ones that speed up decode: (i) *prompt-length discipline*—cap system/RAG context aggressively, since latency tracks context length; (ii) *KV / prefix-cache reuse* across turns to avoid re-prefilling a shared prefix; (iii) *prefill-side quantization* to lift the ∼46 tok/s prefill floor (Section 4.4); and (iv) *skipping optional pipeline stages* on easy queries, since the cheapest token is the one never prefilled or generated—the lever we measure in Section 6. In NeuronAI, budgeting the per-request prompt (capping retrieved context) reduces latency entirely through the prefill term, consistent with this data.

Connection to governed on-device autonomy. On-device "governed bounded autonomy" is latency-gated by prefill: every governance rule, persona instruction, and retrieved passage added to the prompt is paid for at the ∼46 tok/s prefill rate before the model emits a token. The systems bottleneck and the deployment design meet here.

6 A Controlled Lever: Gating the Second Model Pass

If prefill (and, secondarily, decode) dominates cost, the cheapest token is the one never prefilled or generated; the highest-leverage move on CPU is to *skip optional pipeline stages on easy queries*. The single most expensive optional stage in a typical agent pipeline is a second *review/verify* model pass—a whole extra generation over the drafted answer. We test, in a controlled setting, whether that pass earns its latency.

Setup. On the same machine (Table 1), warm Qwen2.5-7B (Q4), we answer 8 factual queries single-pass, then two-pass (draft + a review/revise pass), one run each at temperature 0 (Table 7). For *quality* we ask an LLM judge [18]—run in *both* orders per query to cancel position bias—whether the revised answer beats the single-pass one.

Result. The review pass raises latency **2.41**× (11.96 → 28.79 s) while improving the answer in only **1 of 8** cases (tied in 7, worse in none). It more than doubles latency for a quality gain undetectable on standard factual queries, so gating it to only the hardest queries is a near-free latency win. (The judge is an LLM on relatively easy prompts; harder or generative queries may benefit more, which is why a deployment should retain the pass for its hardest tier.)

Deployment. In NeuronAI this lever sits inside a three-lane *effort router* (reflex / standard / deep) that routes cognitive *depth*—how many pipeline stages run—rather than model size, keeping a ≥7B model on every path. The review pass is enabled only on the deep lane. Lane assignment is a pure-heuristic, model-free classifier (Appendix A), so routing itself adds no model call.

Table 7: Controlled review-pass isolation: single- vs. two-pass latency and order-swapped LLM-judge quality, over 8 matched queries (warm, Qwen2.5-7B Q4, CPU-only). Std is across queries.

Metric	Value
Single-pass latency (s)	11.96±5.49
Two-pass latency (s)	28.79±12.84
Review-pass overhead	2.41×
Revised answer judged better	1 / 8
Tie	7 / 8
Single-pass judged better	0 / 8

7 Limitations

This study centers on one model (Qwen2.5-7B) and one runtime (Ollama/llama.cpp); the cross-family check (Table 4) spans 7B–9B, the quantization sweep (Table 5) spans Q4/Q5/Q8, and we replicate on two x86 CPU vendors (AMD Zen 4 and Intel Comet Lake, Table 6). We report 5 repetitions per point (mean±std); the main remaining extensions are *non-x86* (ARM/mobile) hardware and batched serving. We cap output at 64 tokens; very long generations would raise the decode share. We do not evaluate batched multi-user serving or GPU offload. Prefill is read directly from the runtime's counter, which excludes tokenization/setup overhead folded into total latency. The review-pass lever (Section 6) is measured on 8 factual queries with an LLM judge; a broader query set (harder and generative prompts) and human labels, and a full per-lane latency re-run of the deployed router, remain future work.

8 Conclusion & Future Work

On CPU-only on-device inference, prefill—not decode—is the dominant latency cost, exceeding 90% at realistic prompt lengths and reaching 95% at 2,333 tokens. This inverts GPU-era intuition and reorients edge-LLM optimization toward prompt-size and KV-reuse rather than decode speed. A secondary finding—that prefill speed is non-monotonic in quantization, with the 8-bit build fastest—makes quant choice a prefill-side lever in its own right. Acting on the finding pays off: gating an extra review model pass to only the hardest queries cuts latency 2.4× with no measurable quality loss. On CPU, the cheapest token is the one never prefilled or generated. The prefill-dominance and quant findings reproduce across two x86 CPU vendors (AMD and Intel). Future work: non-x86 (ARM/mobile) hardware, a broader quality evaluation of the routing lever, and direct evaluation of prefill-specific optimizations.

Reproducibility

All harnesses (`prefill_bench.py`, `model_size_sweep.py`, `quant_sweep.py`, `review_pass_experiment.py`, and `cross_hw_sweep.py` for the second machine) and raw per-run JSON/JSONL are released at `https://github.com/bapista/cpu-llm-latency` (directory `data/`), and mirrored in the arXiv ancillary files. The quantization sweep supports a `REVERSE=1` mode that reruns it in reversed model order for the thermal-confound check.

A Effort-Router Lane Classifier

Algorithm 1 gives the model-free classifier and hot-affinity model arbitration referenced in Section 6. G is the resident 7B generalist; *hot* is the currently loaded model ($\varnothing$ if none).

Algorithm 1 Effort routing: lane classification (ClassifyLane) and hot-affinity model arbitration (RouteEffort).

procedure ClassifyLane(q) ▷ no model call
  if q begins with `/think` or `/deep` **then**
    return DEEP
  end if
  $w \leftarrow$ number of words in q
  if $w \leq 10$ **and** q matches a greeting/ack, identity, or time pattern **then**
    return REFLEX
  end if
  if $w > 80$ **or** q has ≥ 2 question marks **or** q matches a hard-signal pattern **then**
    (legal, debug, prove, analyse, compare, design, ...)
    return DEEP
  end if
  return STANDARD ▷ ambiguous biases here
end procedure

procedure RouteEffort(q, *hot*)
  $\ell \leftarrow$ ClassifyLane(q)
  if $\ell =$ REFLEX **and** q is not a web task **then**
    $m \leftarrow$ *hot* **if** *hot* is a non-embedder **else** G
    return generalist troop, model m, *skip* review/grounding
  end if
  prep $\leftarrow$ CommandPrepare(q) ▷ best-fit specialist troop+model
  if $\ell \neq$ DEEP **and** *hot* $\neq \varnothing$ **and** *hot* is a non-embedder **and** *prep.model* $\neq$ *hot* **then**
    prep.model $\leftarrow$ *hot* ▷ avoid a ∼65 s reload
  end if
  prep.review $\leftarrow (\ell =$ DEEP$)$ ▷ review pass: Deep only
  return *prep*
end procedure

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134, 2024.

[2] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2312.11514.

[3] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[4] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[5] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

[6] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2023.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[10] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Tijmen Blankevoort, and Vikas Chandra. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *International Conference on Machine Learning (ICML)*, 2024.

[11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[12] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. arXiv:2311.18677.

[13] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] Haihao Shen, Hanwen Chang, Bo Dong, Yu Luo, and Hengyu Meng. Efficient LLM inference on CPUs. *arXiv preprint arXiv:2311.00502*, 2023.

[15] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023.

[16] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088*, 2024.

[17] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. PowerInfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024.

[18] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[19] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. arXiv:2401.09670.